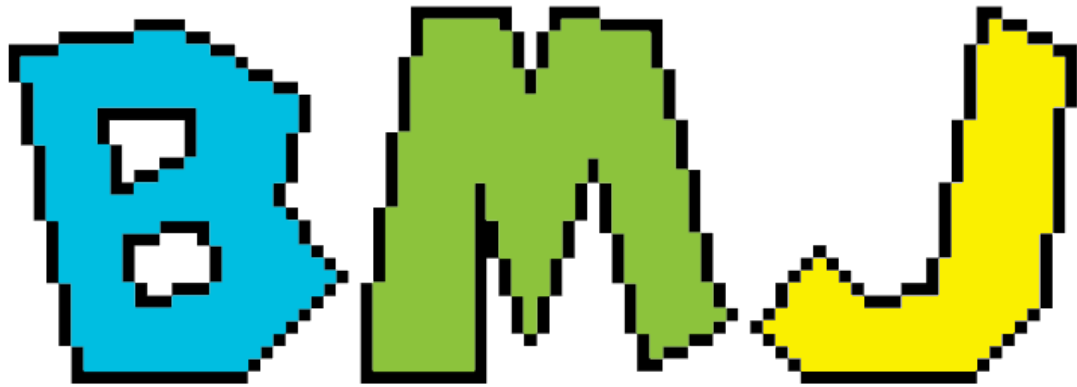




Incidents

Incident Tracker
Comp 229 Section 403 Group Project
BMJ Incidents
<https://github.com/herowkin/BMJ---incidents>
Braiden Jensen, Mason Kerr, & Irhagi Junior Muderhwa
Version 0.1

Table of Contents

- Table of contents: Page 1
- Overview: Page 2
- API: Page 3
- Wireframes: Page 4
- Screenshots: Page 8 (Not yet added)

Overview

For our project, we decided to create an incident management system that allows regular users to post tickets, and admin users to respond to them.

Users are required to login. There are two types of users, regular users, who can create tickets, and view the tickets they have created, and admin users, who can view all tickets, and edit tickets assigned to themselves. They can also edit unassigned tickets to assign them to themselves

Users consist of 4 fields:

- A name, chosen when the user is registered. Names are mandatory.
- An email, chosen when the user is registered. Emails are required to be unique. Emails are required to login Emails are mandatory.
- A password, chosen when the user is registered. Passwords are required to login. Passwords are mandatory
- A role, either USER, for a regular user, or ADMIN, chosen for admin users. If a role is not chosen USER will be assigned by default.

Incidents consist of 7 fields:

- Title, consisting of a short overview of the incident. Titles are mandatory.
- Description, consisting of a longer description of the incident. Descriptions are mandatory
- Category, describing the nature of the incident. By default, incidents are in the category 'General'
- Status, Describing how a ticket is being handled by an ADMIN. Tickets can be 'Open', 'In Progress', 'Resolved', or 'Closed'. By Default, tickets are Open
- Priority, describing the importance of a ticket. Tickets can be 'Low', 'Medium', 'High', or 'Critical' priority. By default, tickets are 'Medium' priority.
- CreatedBy: Automatically generated, referring to the user who created the ticket
- assignedTo: Refers to the ADMIN user who is assigned the ticket.

API

Auth

- POST /api/auth/register { name, email, password }
- POST /api/auth/login { email, password }
- GET /api/auth/me (Bearer token)

Incidents

- GET /api/incidents (query: page, limit, status, priority, q)
- GET /api/incidents/:id
- POST /api/incidents { title, description, category, priority }
- PATCH /api/incidents/:id (e.g., { status, priority, assignedTo })
- DELETE /api/incidents/:id (admin or creator)

Wireframe

Login page, leads to Landing page on successful login.

The wireframe illustrates a login page layout. It features a large rectangular container at the top, centered with the text "Logo/Title". Below this container, three smaller rectangular boxes are stacked vertically and centered. The top box is labeled "Username", the middle box is labeled "Password", and the bottom box is labeled "Login Button".

Landing Page, leads to submit ticket page, and view ticket page

Create a ticket

Search Tickets

Ticket ID

View Tickets

Create ticket page

Title
Description
Category
Priority
Submit

View ticket page

Ticket 1

Title:

Description

Category

Status

Priortiy

Created By Assigned To

If a search returns multiple tickets, they are shown in order, requiring scrolling

Screenshots:

Login:

The screenshot shows a REST client interface with a POST request to `http://localhost:5000/api/auth/login`. The request body is a JSON object containing email and password. The response is a 200 OK status with a JSON body containing a token and user information.

```
POST http://localhost:5000/api/auth/login
```

JSON Content

```
1 {
2   "email": "admin@bmj.local",
3   "password": "Admin@123"
4 }
```

Status: 200 OK Size: 284 Bytes Time: 136 ms

Response

```
1 {
2   "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6IjY5MGZlZjBmZDM5ZmUxYmRlbnZldmZlUwMiIsIm1hdCI6MTc2Mjc2MywzZXhwIjoxNzYzMzYzNTYzZmUwYmRlbnZldmZlUwMiIsInR5cCI6IkpXVCJ9.UBG81kR5V6nT-E5Y_u0fNBAERVjRWJYUN7joDNkZf5Q",
3   "user": {
4     "id": "690fef0fd39fe1bde77fee02",
5     "name": "BMJ Admin",
6     "email": "admin@bmj.local",
7     "role": "admin"
8   }
9 }
```

Create Incident:

The screenshot shows a REST client interface with a POST request to `http://localhost:5000/api/incidents`. The request body is a JSON object containing incident details. The response is a 201 Created status with a JSON body containing the created incident details.

```
POST http://localhost:5000/api/incidents
```

JSON Content

```
1 {
2   "title": "Test incident",
3   "description": "Example of an incident",
4   "category": "Fake",
5   "priority": "Medium"
6 }
```

Status: 201 Created Size: 347 Bytes Time: 26 ms

Response

```
1 {
2   "title": "Test incident",
3   "description": "Example of an incident",
4   "category": "Fake",
5   "status": "Open",
6   "priority": "Medium",
7   "createdBy": {
8     "_id": "690fef0fd39fe1bde77fee02",
9     "name": "BMJ Admin",
10    "email": "admin@bmj.local"
11  },
12  "assignedTo": null,
13  "_id": "69111035199b0d8e2807240c",
14  "createdAt": "2025-11-09T22:05:41.893Z",
15  "updatedAt": "2025-11-09T22:05:41.893Z",
16  "__v": 0
17 }
```

Retrieve Incident:

The screenshot displays a REST client interface with a GET request to `http://localhost:5000/api/incidents/69111035199b0d8` and its corresponding JSON response.

Request Details:

- Method: GET
- URL: `http://localhost:5000/api/incidents/69111035199b0d8`
- Tab: Body¹

Request Body (JSON):

```
1 {
2   "title": "Test incident",
3   "description": "Example of an incident",
4   "category": "Fake",
5   "priority": "Medium"
6 }
```

Response Details:

- Status: 200 OK
- Size: 347 Bytes
- Time: 7 ms
- Tab: Response

Response Body (JSON):

```
1 {
2   "_id": "69111035199b0d8e2807240c",
3   "title": "Test incident",
4   "description": "Example of an incident",
5   "category": "Fake",
6   "status": "Open",
7   "priority": "Medium",
8   "createdBy": {
9     "_id": "690fef0fd39fe1bde77fee02",
10    "name": "BMJ Admin",
11    "email": "admin@bmj.local"
12  },
13   "assignedTo": null,
14   "createdAt": "2025-11-09T22:05:41.893Z",
15   "updatedAt": "2025-11-09T22:05:41.893Z",
16   "__v": 0
17 }
```